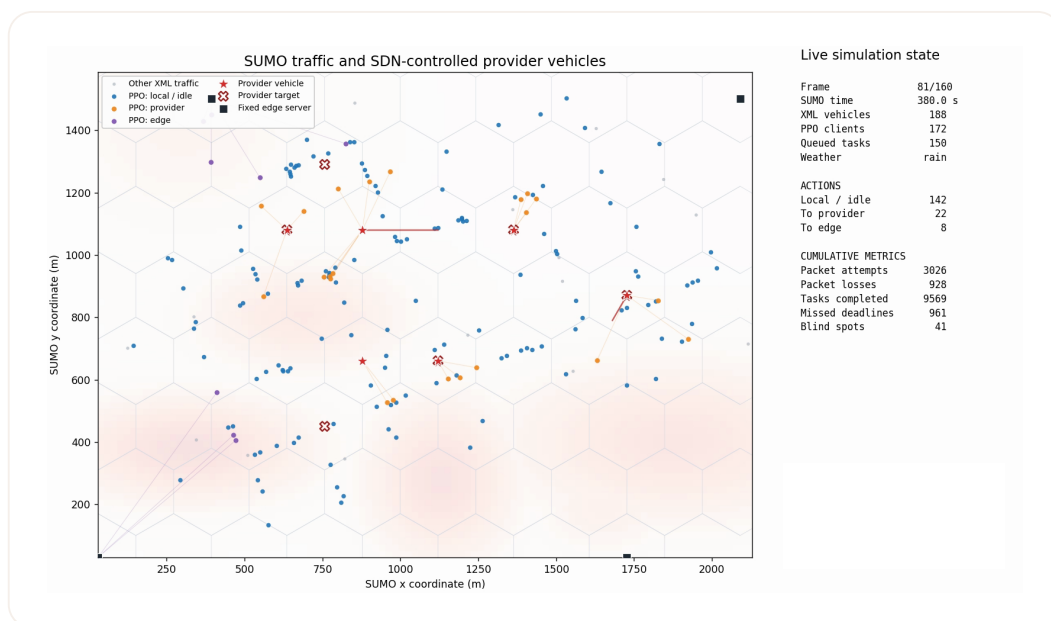


طراحی چارچوب مبتنی بر شبکه SDN برای استقرار ماشین ارائه دهنده و بارسپاری وظایف



حرکت خودروها و پروایدرها در جدول

پروژه سیستم‌های بی‌درنگ
پرهام رضائی ۴۰۰۱۰۸۵۴۷

فهرست مطالب

۱	خلاصه	۱
۱	داده‌ها	۲
۱	۱.۲ ویژگی‌های فایل مسیر	۱
۲	۲.۲ دو مقیاس زمانی	۲
۲	۳ شبکه شش ضلعی و قراردعی	۲
۲	۱.۳ تبدیل نقشه به شش ضلعی	۲
۲	۲.۳ سرورهای ثابت لبه	۲
۳	۳.۳ قراردعی سرویس‌دهنده‌ها به صورت کانوکس	۳
۳	۴ وظایف و مدل لینک و پیش‌بینی کالمن	۳
۳	۱.۴ تولید وظیفه در خودروها	۳
۴	۲.۴ توان پردازشی مقصدها	۴
۴	۳.۴ ریسک مکانی و اتلاف بسته	۴
۴	۴.۴ فیلتر کالمن	۴
۵	۵ فضای الگوریتم آرال	۵
۵	۱.۵ فضای عمل مشترک مقصد و مسیر	۵
۵	۲.۵ ویژگی‌های مدل	۵
۶	۳.۵ ریوارد	۶
۶	۶ معماری و ترین	۶
۶	۱.۶ اکتور شرطی براکشن	۶
۶	۲.۶ اکسپلوریشن و اپیزودها	۶
۷	۳.۶ تابع هدف PPO	۷
۷	۷ چرخه اجرا	۷
۸	۸ توضیح فایل‌ها	۸
۸	۱.۸ config.py	۸
۸	۲.۸ mobility.py	۸

۹	hexgrid.py	۳.۸
۹	network.py	۴.۸
۹	kalman.py	۵.۸
۱۰	placement.py	۶.۸
۱۰	environment.py	۷.۸
۱۱	ppo.py	۸.۸
۱۱	evaluation.py	۹.۸
۱۱	plotting.py	۱۰.۸
۱۲	cli.py و ورودی بسته	۱۱.۸
۱۲	پیکربندی، اجرا و بازتولید	۹
۱۲	سناریوهای اصلی	۱.۹
۱۳	مقایسه نهایی	۱۰
۱۴	نتایج	۱۱
۱۴	۱.۱۱ اتلاف بسته، تاخیر، مهلت و نقطه کور	
۱۵	۲.۱۱ مکان ارائه‌دهندگان در سناریوی ۲۰ خودرو	
۱۵	۳.۱۱ مکان ارائه‌دهندگان در سناریوی ۵۰ خودرو	
۱۶	۴.۱۱ همگرایی PPO	
۱۶	۱۲ ویدیوی گام‌به‌گام	
۱۷	۱۳ جمع‌بندی	

۱ خلاصه

در این پروژه یک شبیه‌سازی شبکه محاسبات لبه‌ای خودروها در دو مقیاس زمانی را انجام دادیم. تعدادی سرور لبه داریم و تعدادی خودروی خدمات‌رسان. این خدمات‌رسان‌ها برحسب یک مسئله محدب تصمیم‌گیری در هر مقیاس زمانی بزرگ به یکی از نقاط نقشه می‌روند. این نقشه توسط یک شش ضلعی بندی فضای مسئله ساخته شده است. پس از انتخاب مرکز شش ضلعی مدنظر، هر خودروی سرویس‌دهنده با سرعت ثابتی در هر لحظه به سمت آن حرکت می‌کند.

حرکت مشتری‌ها از نمونه SUMO خوانده می‌شود. مختصات، سرعت، شناسه و زاویه هر خودرو در هر گام از همان فایل گرفته شده و هیچ مسیر مصنوعی برای مشتری‌ها تولید نمی‌شود. از آنجا که فایل SUMO شامل وظایف محاسباتی، وضعیت آب‌وهوا، اندازه‌گیری رادیویی یا نقشه اتلاف بسته نیست، این اجزا را با مدل‌های تصادفی شبیه‌سازی کرده‌ام. مشتری‌ها تسک‌های ایجادشده خود را با استفاده از یک مدل PPO اجرا می‌کنند که هنگام یادگیری، کریستیک به داده‌های گلوبال و اکتور مشتری‌ها به داده‌های لوکال دسترسی دارد و این همان CTDE مدنظر را می‌دهد.

در نتیجه به طور کلی جزییات طراحی به شکل زیر است.

بخش	روش پیاده‌سازی
تحرك مشتری‌ها	خواندن مستقیم تمام گام‌ها، مختصات و سرعت‌ها از فایل XML
نقشه	محاسبه کمینه و بیشینه مختصات در کل فایل و ساخت شبکه شش ضلعی
خودروهای خدمت‌رسان	تخصیص محدب به مراکز شش ضلعی متمایز و حرکت محدود به سرعت
سرورهای ثابت لبه	به طور قطعی و در دورترین نقاط قرار داده می‌شوند
بارسپاری وظایف	انتخاب همزمان مقصد و نوع مسیر با CTDE-PPO و ماسک عمل
کیفیت لینک	مدل اتلاف وابسته به فاصله، هوا، ریسک مکانی و سرعت به همراه فیلتر کالمن
ارزیابی	چهار ترکیب قراردادی محدب/تصادفی و انتخاب PPO/حریصانه
ارزیابی را به جای دو مدل، چهار مدل کردم تا تاثیر هر بخش درست مشخص شود.	

۲ داده‌ها

۱.۲ ویژگی‌های فایل مسیر

تابع `TrafficTrace.from_xml()` فایل خودروها را به‌صورت جریانی با `iterparse` می‌خواند. زمان‌ها باید صعودی باشند و هر خودرو باید حداقل شناسه، مختصات و سرعت معتبر داشته باشد. همزمان با خواندن، کمینه و بیشینه مختصات در تمام گام‌ها محاسبه می‌شود؛ بنابراین نقشه فقط به یک اپیزود یا یک زیرمجموعه از خودروها وابسته نیست.

مقدار	مشخصه
۱۰۰۱ گام، از ثانیه ۳۰۰ تا ۱۳۰۰	تعداد گام‌های زمانی
از ۵۰/۲۹ تا ۶۴/۲۱۲۸ متر	بازه x
از ۳۸/۳۰ تا ۶۸/۱۵۸۶ متر	بازه y
۱۷۲ تا ۲۱۳ خودرو	خودروهای فعال در هر گام
۱۲۴۳ شناسه	تعداد شناسه‌های یکتا

هیچ شناسه‌ای در تمام ۱۰۰۱ گام حضور نداشت. وقتی خودرویی خارج می‌شود، یکی از خودروهایی که قبلاً استفاده نشده است جایگزین آن می‌شود و وارد سیستم می‌شود. این کار در تابع `client_frame()` انجام شده است. صف و فیلتر کالمن مربوط به هر خودرو نیز هنگام خروج آن حذف می‌شود.

۲.۲ دو مقیاس زمانی

ما از فایل SUMO شناسه، زمان، سرعت و مکان را می‌گیریم. سپس در مقیاس بزرگ، ریسک و تقاضا را حساب کرده و تخصیص کانوکس به مرکز شش ضلعی‌ها را انجام می‌دهیم. از آن برای حالت شبکه، مکان پروایدرها و بار پردازشی آن‌ها استفاده می‌کنم. همچنین از مکان‌های موجود در SUMO و مقادیر مقیاس بزرگ، یعنی جاگیری‌ها، استفاده می‌کنم تا در مقیاس کوچک با استفاده از صف، خروجی فیلتر کالمن و ماسک‌ها، الگوریتم PPO را انجام دهیم. در نهایت از آن به خروجی، اتلاف، تاخیر و نمودارهای مدنظر می‌رسیم.

قراردهی ارائه‌دهندگان هر ۲۰ ثانیه انجام می‌شود، در حالی که تصمیم بارسپاری وظیفه و حرکت خودروها در هر گام یک‌ثانیه‌ای اجرا می‌شود. هنگام آموزش، اکثراً فقط مشاهده محلی را می‌بیند، اما کریتیک اطلاعات جمعی مانند تراکم، میانگین صف و بار گره‌ها را دریافت می‌کند.

۳ شبکه شش ضلعی و قراردهی

۱.۳ تبدیل نقشه به شش ضلعی

کلاس HexGrid یک شبکه شش ضلعی به صورت عمودی می‌سازد. اگر شعاع شش ضلعی r باشد، فاصله افقی مراکز برابر $\sqrt{3}r$ و فاصله عمودی ردیف‌ها $1/2r$ است. ردیف‌های زوج و فرد نیم فاصله افقی جابه‌جا می‌شوند. شعاع پیش فرض ۱۴۰ متر است و فقط مراکزی که داخل مستطیل کمینه/بیشینه فایل قرار دارند، نقاط هدف سرویس‌دهنده‌ها می‌شوند. تابع `nearest_indices()` از `ckdTree` استفاده می‌کند تا مکان پیوسته هر ارائه‌دهنده را به نزدیک‌ترین خانه شش ضلعی نسبت دهد. تابع `vertices()` نیز شش راس چندضلعی را برای ترسیم نقشه و نقشه حرارتی تولید می‌کند.

۲.۳ سرورهای ثابت لبه

تابع `fixed_edge_positions()` جاده‌ی قطعی دورترین نقطه را اجرا می‌کند. اولین سرور نزدیک گوشه با کمترین مختصات قرار می‌گیرد. سپس در هر تکرار مرکزی انتخاب می‌شود که کمترین فاصله‌اش از مجموعه سرورهای فعلی بیشینه باشد. در نتیجه، سرورهای ثابت بدون استفاده از اطلاعات آینده ترافیک در سطح نقشه پخش می‌شوند.

۳.۳ قراردعی سرویس‌دهنده‌ها به صورت کانوکس

برای مرکز شش ضلعی h و مشتری i ، ضریب پوشش موفق a_{hi} از فاصله، شدت هوا و ریسک نقطه میانی لینک محاسبه می‌شود و اگر خارج از برد ارائه‌دهنده باشد صفر است. تقاضای مشتری از طول صف و تعداد سایکل‌های وظیفه سر صف ساخته می‌شود:

$$d_i = 1 + \frac{|Q_i|}{Q_{\max}} + \frac{\text{سایکل‌های تسک اول}}{2/2 \times 10^9}.$$

متغیر x_{ph} سهم تخصیص ارائه‌دهنده p به مرکز h و u_i سهم تقاضای پوشش داده‌شده مشتری i است. مسئله زیر حل می‌شود:

$$\begin{aligned} \max_{x,u} \quad & \sum_i d_i u_i - \lambda_m \sum_{p,h} \frac{\|c_h - z_p\|}{R_p} x_{ph}, \\ \sum_h x_{ph} = 1, \quad & \sum_p x_{ph} \leq 1, \quad 0 \leq x_{ph} \leq r_{ph}, \\ 0 \leq u_i \leq 1, \quad & u_i \leq \sum_h a_{hi} \sum_p x_{ph}. \end{aligned}$$

r_{ph} برای خانه‌ای که در یک بازه قابل دسترس نیست صفر می‌شود. قید $u_i \leq 1$ از پوشش دادن تکراری مشتری جلوگیری می‌کند. مسئله خطی و محدب است. چون جواب رگولارایز کردن ممکن است کسری باشد، یک تطبیق دو بخشی با بیشینه وزن آن را به مراکز متمایز و قابل دسترس گرد می‌کند؛ یعنی همان مسئله linear sum assignment.

تابع `move_toward_targets()` ارائه‌دهنده را در راستای هدف حرکت می‌دهد، اما جابه‌جایی هر ثانیه از ۲۵ متر بیشتر نمی‌شود. در نتیجه خودروها به هدف تله‌پورت نمی‌شوند.

بیس‌لاین

RandomProviderPlacer نیز فقط مراکز متمایز و قابل دسترس را انتخاب می‌کند، اما تقاضا، ریسک و آب‌وهوا را در امتیازدهی وارد نمی‌کند. این روش بیس‌لاین مقایسه است.

۴ وظایف و مدل لینک و پیش‌بینی کالمن

۱.۴ تولید وظیفه در خودروها

در هر گام و برای هر مشتری، با احتمال $0/7$ یک وظیفه جدید ایجاد می‌شود. این فرآیند برنولی مستقل باعث می‌شود میانگین نرخ ورود تقریباً $0/7$ وظیفه بر ثانیه برای هر خودرو باشد. تابع `sample_task()` مقادیر زیر را نمونه‌گیری می‌کند:

ویژگی	توضیح
اندازه ورودی	یکنواخت بین ۰/۶ تا ۴ مگابیت؛ تعیین‌کننده زمان ارسال
چرخه پردازشی	یکنواخت بین ۰/۴ تا ۲/۲ گیگاسیکل؛ تعیین‌کننده زمان پردازش
مهلت نسبی	یکنواخت بین ۱/۵ تا ۵ ثانیه
زمان ایجاد	زمان فعلی فایل SUMO برای محاسبه تاخیر و زمان انتظار

صف هر خودرو حداکثر ۱۲ وظیفه دارد و در هر گام فقط وظیفه سر صف اجرا می‌شود. اگر صف پر باشد، ورود جدید هم افت صف و هم نقض مهلت محسوب می‌شود. ارسال ناموفق وظیفه را در صف نگه می‌دارد، مگر اینکه با گذشت گام بعدی مهلت آن تمام شده باشد.

۲.۴ توان پردازشی مقصدها

توان محاسباتی ثابت هر کدام به شکل زیر است:

$$f_{\text{local}} = 0.8 \text{GHz}, \quad f_{\text{provider}} = 12 \text{GHz}, \quad f_{\text{edge}} = 25 \text{GHz}.$$

زمان پردازش محلی برابر $\text{cycles}/f_{\text{local}}$ است. برای ارائه‌دهنده و سرور لبه، تعداد وظایف موفق که همزمان به همان گره تخصیص یافته‌اند به عنوان ضریب شلوغی پردازنده در زمان پردازش ضرب می‌شود.

۳.۴ ریسک مکانی و اتلاف بسته

SpatialRiskMap چند نقطه گاوسی را با سید ثابت داخل حدود نقشه می‌سازد. احتمال اتلاف یک لینک تابعی از فاصله نرمال‌شده، آب‌وهوا، ریسک نقطه میانی و سرعت فرستنده است:

$$\text{logit}(p) = -4/2 + 5/\sqrt{\frac{d}{R}} + 1/15w + 1/65s + 0.010v.$$

اگر فاصله از برد مجاز بیشتر باشد، احتمال اتلاف یک و عمل مربوطه نامعتبر است. برای مسیر چندپرسی:

$$p_{\text{path}} = 1 - \prod_k (1 - p_k).$$

نرخ داده از یک تقریب محدودشده $B \log_2(1 + \text{SNR})$ با پهنای باند ۱۰ مگاهرتز محاسبه می‌شود. زمان ارسال هر پرس برابر اندازه وظیفه تقسیم بر نرخ همان پرس است.

۴.۴ فیلتر کالمن

الگوریتم RL احتمال واقعی را نمی‌بیند. ابتدا نویز گاوسی به احتمال واقعی افزوده می‌شود، سپس برای هر کلید خودرو/عمل مقصد یک فیلتر کالمن اسکالر مستقل نگهداری می‌شود:

$$P_t^- = P_{t-1} + Q, \quad K_t = \frac{P_t^-}{P_t^- + R},$$

$$\hat{p}_t = \hat{p}_t^- + K_t(z_t - \hat{p}_t^-), \quad P_t = (1 - K_t)P_t^-.$$

مقادیر پیش‌فرض واریانس فرایند و اندازه‌گیری به ترتیب ۰/۰۱۵ و ۰/۰۴ هستند. خروجی در بازه $[0, 1]$ کلیپ می‌شود و به الگوریتم حریصانه یا RL داده می‌شود.

۵ فضای الگوریتم آرال

۱.۵ فضای عمل مشترک مقصد و مسیر

برای P ارائه‌دهنده و E سرور لبه، تعداد اعمال برابر است با:

$$|A| = 1 + 2P + 2E.$$

عمل صفر پردازش محلی است. برای هر ارائه‌دهنده دو عمل ارسال مستقیم و ارسال با میانی خودرویی، و برای هر سرور لبه دو عمل مستقیم و میانی از طریق ارائه‌دهنده وجود دارد. بنابراین سناریوی ۲۰ خودرو ۱۱ عمل و سناریوی ۵۰ خودرو ۲۱ عمل دارد.

مقصد	مسیرها	شرط اعتبار
خود خودرو	محلی	همیشه معتبر؛ برای صف خالی تنها عمل معتبر است
ارائه‌دهنده p	مستقیم، میانی خودرویی	تمام پرش‌های لازم باید داخل برد باشند
سرور لبه e	مستقیم، میانی ارائه‌دهنده	لینک مستقیم یا هر دو پرش مشتری-ارائه‌دهنده-لبه معتبر باشند

۲.۵ ویژگی‌های مدل

مشاهده محلی هر مشتری از هفت ویژگی پایه و سه ویژگی برای هر عمل تشکیل می‌شود.

گروه	ویژگی‌ها
صف و وظیفه	طول صف نرمال، اندازه داده، چرخه لازم، سهم مهلت باقی‌مانده و نسبت زمان سپری‌شده به مهلت
تحرک و زمینه	سرعت واقعی نرمال‌شده و شدت آب‌وهوا
برای هر عمل	اتلاف تخمینی کالمن، زمان ارسال تقسیم بر مهلت، زمان پردازش تقسیم بر مهلت

ابعاد دقیق ورودی‌ها:

سناریو	عمل	مشاهده	مرکزی	ورودی کریٹیک
۲ / ۳ / ۲۰	۱۱	$7 + 3(11) = 40$	$4 + 3 + 2 = 9$	۴۹
۴ / ۶ / ۵۰	۲۱	$7 + 3(21) = 70$	$4 + 6 + 4 = 14$	۸۴

۳.۵ ریوارد

برای ارسال دوردست، اکتور در هر تلاش جریمه ریسک $\lambda_{\text{loss}} \hat{p}$ می‌گیرد. اگر ارسال واقعا شکست بخورد، یک هزینه تاخیر یک‌گام و در صورت پایان مهلت جریمه مهلت افزوده می‌شود. برای وظیفه موفق:

$$r_i = -\lambda_d \min\left(\frac{D_i}{T_i}, 4\right) - \lambda_{\text{loss}} \hat{p}_i - \lambda_T \mathbb{I}[D_i > T_i].$$

ضرایب پیش‌فرض تاخیر، اتلاف و مهلت به ترتیب ۱، ۰/۷۵ و ۴ هستند.

معیارهای گزارش شده که از دستور پروژه برداشته شده‌اند عبارت‌اند از:

- نسبت اتلاف بسته: تعداد ارسال‌های دوردست ناموفق تقسیم بر تعداد تلاش‌ها
- میانگین تاخیر: میانگین تاخیر برای وظایف تکمیل شده
- نقض مهلت: تکمیل دیرهنگام، انقضا، سرریز صف یا خروج خودرو با وظیفه ناتمام
- نقطه کور: مشتری‌ای که در یک گام هیچ عمل دوردست قابل دسترس ندارد
- مکان ارائه‌دهنده: توزیع زمانی حضور در شش ضلعی‌ها و میانگین مکان هر ارائه‌دهنده

۶ معماری و ترین

۱.۶ اکتور شرطی بر اکشن

هفت ویژگی پایه با سه ویژگی همان عمل ترکیب می‌شوند؛ پس ورودی امتیازدهنده همیشه ۱۰ بعدی است:

$$10 \rightarrow 128 \rightarrow \tanh \rightarrow 128 \rightarrow \tanh \rightarrow 1.$$

خروجی این شبکه برای تمام اعمال تکرار می‌شود.

کریتیک ورودی مرکزی ۴۹ یا ۸۴ بعدی را دریافت می‌کند:

$$n_c \rightarrow 128 \rightarrow \tanh \rightarrow 128 \rightarrow \tanh \rightarrow 1.$$

وزن لایه‌های خطی به صورت متعامد و بایاس‌ها صفر مقداردهی می‌شوند.

۲.۶ اکسپلوریشن و اپیزودها

در هر به‌روزرسانی ۹۶ گام برای تمام مشتری‌ها جمع می‌شود. پاداش‌ها بر ۴ تقسیم می‌شوند، ارزش گام بعد بوت‌استرپ می‌شود و تابع اپوینج به صورت معکوس زمانی محاسبه می‌گردد:

$$\delta_t = r_t + \gamma V(s_{t+1})(1 - d_t) - V(s_t),$$

$$A_t = \delta_t + \gamma \lambda (1 - d_t) A_{t+1}.$$

۳.۶ تابع هدف PPO

نسبت احتمال پالیسی جدید به قدیم $\rho_t = \exp(\log \pi_\theta - \log \pi_{\text{old}})$ است. لاس پالیسی:

$$L_\pi = -\mathbb{E} [\min (\rho_t A_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) A_t)].$$

آنتروپی هم اضافه می‌شود:

$$L = L_\pi + c_v L_V - c_e H(\pi).$$

پارامتر	مقدار	کارکرد
تعداد به‌روزرسانی / طول رول‌اوت	۹۶ / ۱۰۰	مقدار تجربه آموزش
دوره / مینی‌بچ	۵۱۲ / ۴	استفاده چندباره و درهم‌ریخته از هر رول‌اوت
نرخ یادگیری	3×10^{-4}	گام بهینه‌ساز Adam

۷ چرخه اجرا

تابع `VehicularEdgeEnv.step()` نقطه اتصال همه مدل‌هاست. ترتیب رخدادها در یک گام به شرح زیر است:

۱. شکل آرایه اعمال و اعتبار هر عمل بررسی می‌شود و اگر نامعتبر باشد، دیفالت روی لوکال قرار می‌گیرد.
۲. برای تمام مشتری‌ها وجود حداقل یک مسیر دوردست بررسی و نقاط کور شمرده می‌شوند.
۳. برای وظیفه سر صف، ارسال محلی مستقیماً موفق و ارسال دوردست با احتمال واقعی لینک نمونه‌گیری می‌شود.
۴. ارسال ناموفق در شمارنده اتلاف ثبت می‌شود و وظیفه، تا زمان انقضا، در صف می‌ماند.
۵. تعداد وظایف موفق به هر ارائه‌دهنده و لبه محاسبه می‌شود.
۶. زمان پردازش با در نظر گرفتن ازدحام، زمان ارسال و مدت انتظار وظیفه ترکیب می‌شود.
۷. وظیفه تکمیل شده حذف و تاخیر، پاداش و نقض مهلت به‌روز می‌شود.
۸. ارائه‌دهندگان حداکثر ۲۵ متر به سمت هدف حرکت می‌کنند.
۹. گام بعدی XML خوانده، خودروهای خارج شده جایگزین و حالت‌های قدیمی حذف می‌شوند.
۱۰. آب‌وهوا با احتمال ۰/۰۶ تغییر، وظایف جدید وارد و هر ۲۰ گام هدف‌های کانوکس محاسبه می‌شوند.
۱۱. مشاهده لوکال، حالت مرکزی و ماسک عمل گام بعد ساخته می‌شود.

نکته این است که همه چیز با NumPy پیاده‌سازی شده و سعی شده موازی‌سازی درست انجام شود تا برای سناریوی سنگین‌تر هم زمان زیادی نگیرد؛ هرچند کل فرآیند یادگیری همچنان چند دقیقه زمان‌بر است.

۸ توضیح فایل‌ها

۱.۸ config.py

تابع	نقش
ScenarioConfig	تعداد مشتری، ارائه‌دهنده و سرور لبه هر سناریو را نگه می‌دارد.
SimulationConfig	پارامترهای هندسه، رادیو، پردازنده، وظیفه، مهلت و پاداش را متمرکز می‌کند.
PPOConfig	ابزارهای شبکه، رول‌اوت، GAE، کلیپ و بهینه‌ساز را نگه می‌دارد.
ExperimentConfig.to_dict()	تنظیمات کامل را برای ذخیره در JSON سریال می‌کند.
_merge_dataclass()	مقادیر YAML را ادغام و کلید ناشناخته را خطا می‌دهد.
load_config()	پیش‌فرض‌ها را می‌سازد و فایل پیکربندی را اعتبارسنجی و بارگذاری می‌کند.

۲.۸ mobility.py

تابع	نقش
TrafficFrame	یک گام خام شامل زمان، شناسه‌ها، مکان، سرعت و زاویه است.
TrafficFrame.index()	نگاشت شناسه به ردیف آرایه را برای نگهداشت مشتری می‌سازد.
ClientFrame	نمای ثابت مشتری‌ها و پرچم خودروهای جایگزین شده را نگه می‌دارد.
TrafficTrace.from_xml()	فایل را جریانی می‌خواند، زمان و صفات را بررسی و حدود جهانی را استخراج می‌کند.
validate_scenario()	بیش‌بودن تعداد مشتری یا طول اپیزود را رد می‌کند.
client_frame()	خودروهای فعال را نگه می‌دارد و جای خالی را قطعی پر می‌کند.
iter_client_frames()	دنباله‌ای پیوسته از قاب‌های مشتری تولید می‌کند.

hexgrid.py ۳.۸

تابع	نقش
HexGrid.from_bounds()	مراکز ردیف‌های شش ضلعی را از حدود فایل و شعاع می‌سازد.
nearest_indices()	نقاط پیوسته را با cKDTree به نزدیک‌ترین مرکز نگاشت می‌کند.
vertices()	شش راس هر خانه را برای ترسیم برمی‌گرداند.
diagonal	قطر مستطیل نقشه را محاسبه می‌کند.

network.py ۴.۸

تابع	نقش
Task	اندازه داده، چرخه، مهلت و زمان ایجاد را نگه می‌دارد.
SpatialRiskMap	نقاط داغ ریسک بزرگ‌گذاری شده را تولید و ارزیابی می‌کند.
LinkModel.packet_loss()	اتلاف را از فاصله، هوا، ریسک و سرعت محاسبه می‌کند.
noisy_measurement()	نویز اندازه‌گیری را پیش از کالمن اضافه می‌کند.
data_rate_bps()	نرخ داده کران‌دار را از فاصله و هوا به دست می‌آورد.
sample_task()	ویژگی‌های وظیفه را از بازه‌های پیکربندی نمونه می‌گیرد.

kalman.py ۵.۸

تابع	نقش
ScalarKalmanFilter.predict()	عدم قطعیت را با واریانس فرایند افزایش می‌دهد.
ScalarKalmanFilter.update()	بهره کالمن، میانگین و واریانس پسین را محاسبه می‌کند.
LinkKalmanBank.estimate()	فیلتر هر کلید لینک را تثبیل می‌سازد و به‌روزرسانی می‌کند.
discard_vehicles()	حالت لینک خودروهای خارج شده را حذف می‌کند.

۶.۸ placement.py

تابع	نقش
<code>fixed_edge_positions()</code>	سرورهای ثابت را با دورترین-نقطه توزیع می‌کند.
<code>_coverage()</code>	ماتریس موفقیت مرکز شش ضلعی به مشتری را می‌سازد.
<code>ConvexProviderPlacer</code>	تابع <code>choose_targets()</code> برنامه خطی را حل و جواب را به مراکز متمایز گرد می‌کند.
<code>RandomProviderPlacer</code>	تابع <code>choose_targets()</code> مراکز تصادفی متمایز و قابل دسترس را انتخاب می‌کند.
<code>move_toward_targets()</code>	جابه‌جایی هر ارائه‌دهنده را به سرعت مجاز محدود می‌کند.

۷.۸ environment.py

تابع	نقش
<code>ActionSpec</code>	نوع مقصد، شماره مقصد و نوع مسیر را توصیف می‌کند.
<code>NetworkMetrics.as_dict()</code>	اتلاف، تاخیر، مهلت، نقطه کور، تکمیل و افت را محاسبه می‌کند.
<code>__init__()</code>	مولدهای تصادفی مستقل، شبکه، ریسک، مدل لینک، قراردعی، ابعاد و آرایه‌های حالت را می‌سازد.
<code>_make_action_specs()</code>	عمل محلی و دو عمل برای هر ارائه‌دهنده و لبه می‌سازد.
<code>reset()</code>	قطعه اپیزود، صف‌ها، هوا و مکان اولیه ارائه‌دهندگان را مقداردهی می‌کند.
<code>_enqueue_arrivals()</code>	ورود برنولی وظایف و ظرفیت صف را اعمال می‌کند.
<code>_placement_demand()</code>	تقاضای قراردعی را از صف و چرخه وظیفه می‌سازد.
تابع	نقش
<code>_calculate_paths()</code>	فاصله‌ها، اتلاف‌ها، رله‌های برتر و ماسک اتصال را برداری محاسبه می‌کند.
<code>_observations()</code>	هفت ویژگی پایه و سه ویژگی هر عمل را همراه کالمن می‌سازد.
<code>_central_state()</code>	میانگین صف، تراکم، هوا، زمان و بار گره‌ها را برای کریتیک اضافه می‌کند.
<code>_transmission_delay()</code>	زمان یک یا دو پرش را جمع می‌کند.
<code>step()</code>	عمل، اتلاف، ازدحام پردازش، صف، پاداش، معیار و گذار را اجرا می‌کند.
<code>_advance()</code>	ارائه‌دهندگان، خودروها، هوا، ورود و بازقراردعی را جلو می‌برد.
<code>_drop_expired_tasks()</code>	وظایف منقضی را حذف و نقض مهلت ثبت می‌کند.

۸.۸ ppo.py

تابع	نقش
<code>ActorCritic.__init__()</code>	اکتور شرطی بر عمل و کریٹیک مرکزی را می‌سازد.
<code>_initialize()</code>	وزن متعادل و بایاس صفر اعمال می‌کند.
<code>_actor_logits()</code>	ویژگی پایه را کنار ویژگی هر عمل می‌گذارد و لگیت می‌سازد.
<code>distribution()</code>	ماسک را بررسی و توزیع دسته‌ای معتبر ایجاد می‌کند.
<code>act()</code>	عمل تصادفی یا قطعی، لگاریتم احتمال و ارزش را می‌دهد.
<code>evaluate_actions()</code>	احتمال، آنتروپی و ارزش را در به‌روزرسانی بازحساب می‌کند.
<code>PPOTrainer.train()</code>	رول‌اوت، GAE، نرمال‌سازی، مینی‌بچ و بهینه‌سازی کلیپ‌شده را اجرا می‌کند.
<code>save()</code>	وزن‌ها، حالت بهینه‌ساز و ابعاد معماری را ذخیره می‌کند.
<code>load_actor_critic()</code>	مدل را از فراداده بازسازی و در حالت ارزیابی قرار می‌دهد.

۹.۸ evaluation.py

تابع	نقش
<code>GreedyOffloader.select()</code>	هر عمل معتبر را با زمان ارسال تعدیل‌شده با تکرار، پردازش و اتلاف امتیاز می‌دهد.
<code>EvaluationResult</code>	میانگین، انحراف معیار، تاریخچه و میانگین مکان ارائه‌دهندگان را نگه می‌دارد.
<code>evaluate()</code>	اپیزودهای هم‌بدر را اجرا، عمل قطعی انتخاب و آمار و تاریخچه‌ها را جمع می‌کند.

۱۰.۸ plotting.py

تابع	نقش
<code>plot_metric_comparison()</code>	چهار نمودار میله‌ای اتلاف، تاخیر، مهلت و نقطه کور را می‌سازد.
<code>_provider_occupancy()</code>	مکان‌ها را به خانه نزدیک نگاشت و درصد حضور ارائه‌دهنده-زمان را محاسبه می‌کند.
<code>plot_provider_locations()</code>	نقشه حرارتی محدب/تصادفی، ستاره میانگین و مربع لبه را رسم می‌کند.
<code>plot_convergence()</code>	پاداش رول‌اوت، زیان سیاست و زیان ارزش را برحسب به‌روزرسانی رسم می‌کند.

۱۱.۸ cli.py و ورودی بسته

تابع	نقش
<code>command_analyze()</code>	فایل مسیر را اعتبارسنجی و آمار آن را چاپ می‌کند.
<code>command_train()</code>	یک سناریو را آموزش و، CSV و نمودار را ذخیره می‌کند.
<code>command_evaluate()</code>	هر چهار روش را ارزیابی می‌کند.
<code>command_run_all()</code>	دو سناریو را آموزش، ارزیابی و تمام خروجی‌ها را تولید می‌کند.
<code>command_video()</code>	سناریوی ویدیو را می‌سازد و رندر را اجرا می‌کند.
<code>main()</code> و <code>build_parser()</code>	آرگومان‌ها، پیکربندی و توزیع زیرفرمان‌ها را مدیریت می‌کنند.
<code>__main__.py</code>	اجرای <code>python -m vecsim</code> را به <code>main()</code> وصل می‌کند.

۹ پیکربندی، اجرا و بازتولید

۱.۹ سناریوهای اصلی

سناریو	مشتري	ارائه‌دهنده	سرور لبه
scenario_20	۲۰	۳	۲
scenario_50	۵۰	۶	۴

اگر خواستید تست کنید، دستور آموزش و بعد ارزیابی این است:

```
XML="/Users/cyberrose/Education/Sharif/Term 10/Real-time systems/Project/
simulation.out.xml"
```

```
python3 -m vecsim --config configs/default.yaml run-all \
--xml "$XML" \
--output outputs
```

برای ویدیو هم:

```
python3 -m vecsim --config configs/default.yaml video \
--xml "$XML" \
--scenario scenario_50 \
--checkpoint outputs/checkpoints/scenario_50_ppo.pt \
--clients 172 --frames 160 --fps 10 \
--output outputs/simulation_172_clients.mp4
```

۱۰ مقایسه نهایی

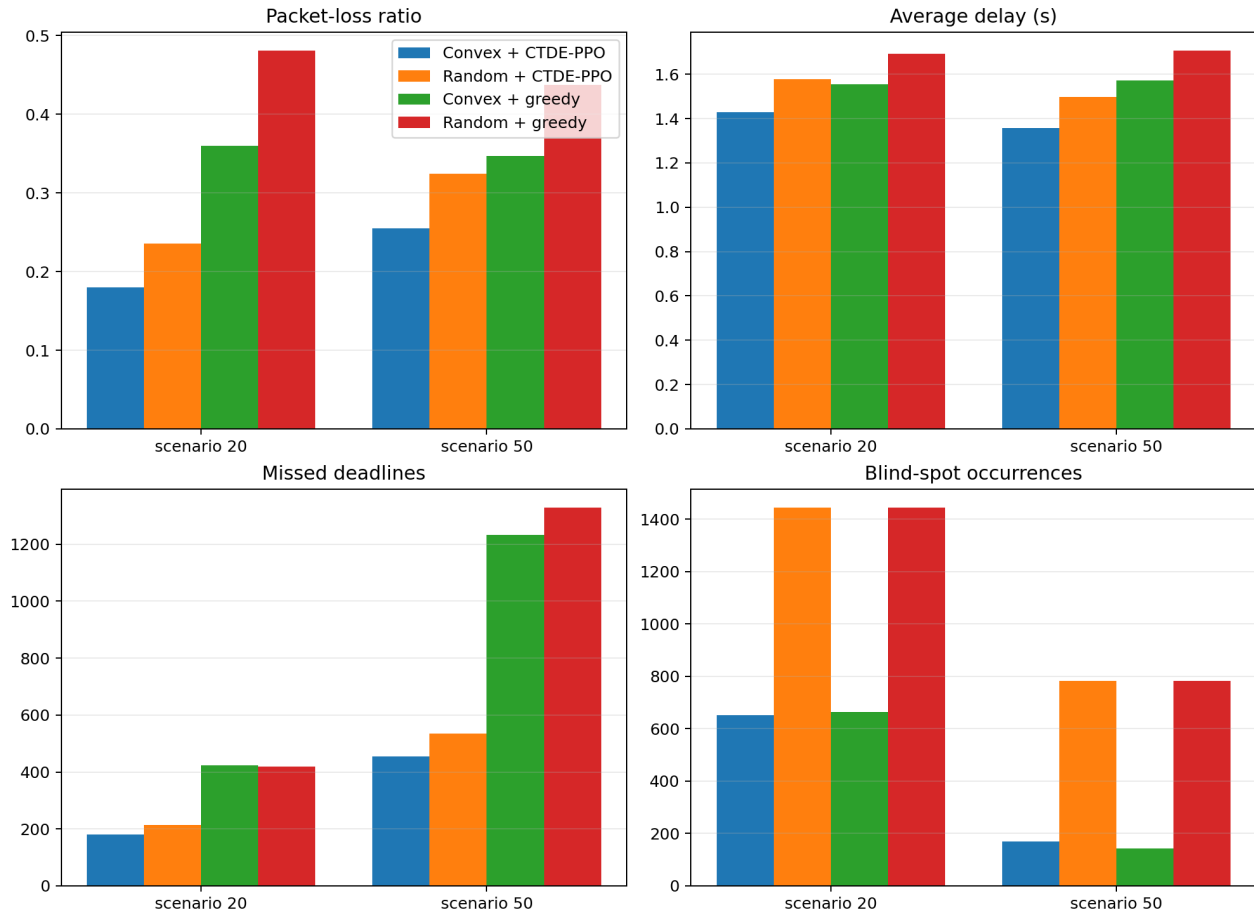
ارزیابی نهایی روی پنج اپیزود با نقاط شروع پخش شده در فایل و سیدهای متناظر انجام شده است:

- قراردعی محدب + بارسیاری CTDE-PPO
- قراردعی تصادفی + CTDE-PPO: سنجش اثر قراردعی؛
- قراردعی محدب + بارسیاری حریصانه: سنجش اثر یادگیری؛
- قراردعی تصادفی + حریصانه: الگوریتم فاز اول.

سناریو	روش	اتلاف	تاخیر	مهلت	نقطه کور
۲/۳/۲۰	محدب + PPO	۱۸۰/۰	۴۲۹/۱	۲/۱۸۰	۸/۶۵۰
۲/۳/۲۰	تصادفی + PPO	۲۳۶/۰	۵۷۸/۱	۴/۲۱۳	۲/۱۴۴۳
۲/۳/۲۰	محدب + حریصانه	۳۶۰/۰	۵۵۵/۱	۴/۴۲۳	۶/۶۶۲
۲/۳/۲۰	تصادفی + حریصانه	۴۸۱/۰	۶۹۳/۱	۲/۴۱۸	۲/۱۴۴۳
۴/۶/۵۰	محدب + PPO	۲۵۵/۰	۳۵۶/۱	۲/۴۵۵	۰/۱۷۰
۴/۶/۵۰	تصادفی + PPO	۳۲۴/۰	۴۹۶/۱	۲/۵۳۶	۲/۷۸۲
۴/۶/۵۰	محدب + حریصانه	۳۴۷/۰	۵۷۱/۱	۴/۱۲۳۳	۰/۱۴۴
۴/۶/۵۰	تصادفی + حریصانه	۴۳۸/۰	۷۰۶/۱	۲/۱۳۲۸	۲/۷۸۲

۱۱ نتایج

۱.۱۱ اتلاف بسته، تاخیر، مهلت و نقطه کور

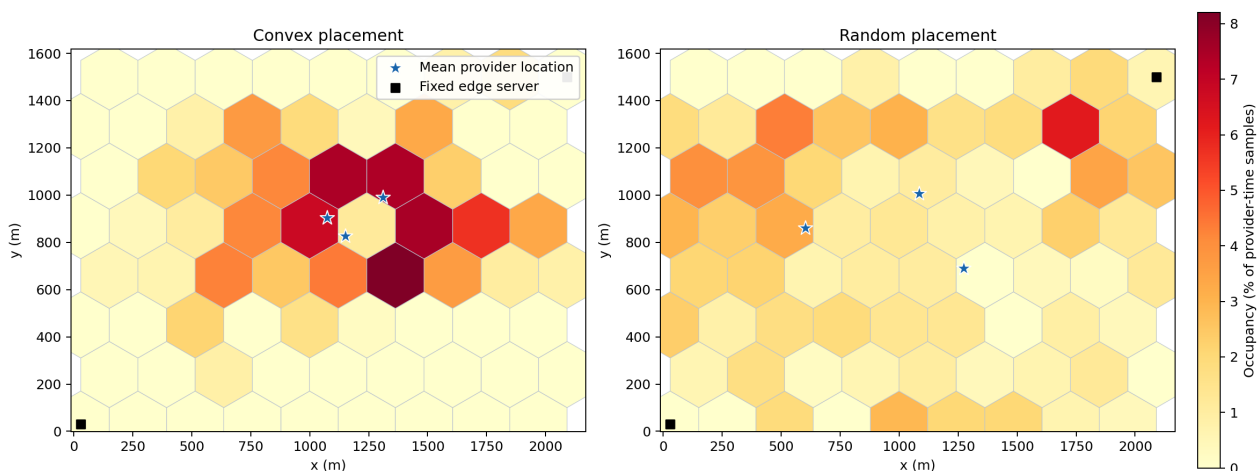


شکل ۱: چهار معیار اصلی شبکه برای دو سناریو و چهار روش

روش محدب + CTDE-PPO در هر دو سناریو کمترین اتلاف، کمترین تاخیر و کمترین نقض مهلت را دارد. نسبت به خط پایه تصادفی + حریصانه، اتلاف بسته، میانگین تاخیر و نقض مهلت به‌طور قابل توجهی کاهش یافته است.

۲.۱۱ مکان ارائه‌دهندگان در سناریوی ۲۰ خودرو

Provider occupancy heat map and mean locations — scenario 20

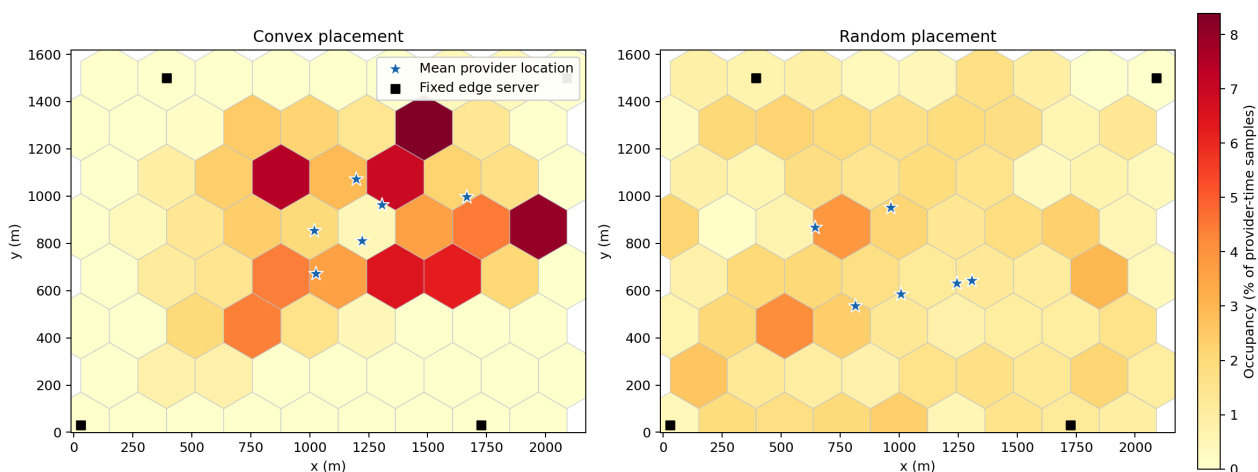


شکل ۲: نقشه حرارتی حضور ارائه‌دهندگان و مکان میانگین در سناریوی ۲۰/۳/۲۰

در قراردادی محدب، حضور در ناحیه مرکزی ترافیک متمرکز است، در حالی که قراردادی تصادفی پراکندگی بیشتری دارد.

۳.۱۱ مکان ارائه‌دهندگان در سناریوی ۵۰ خودرو

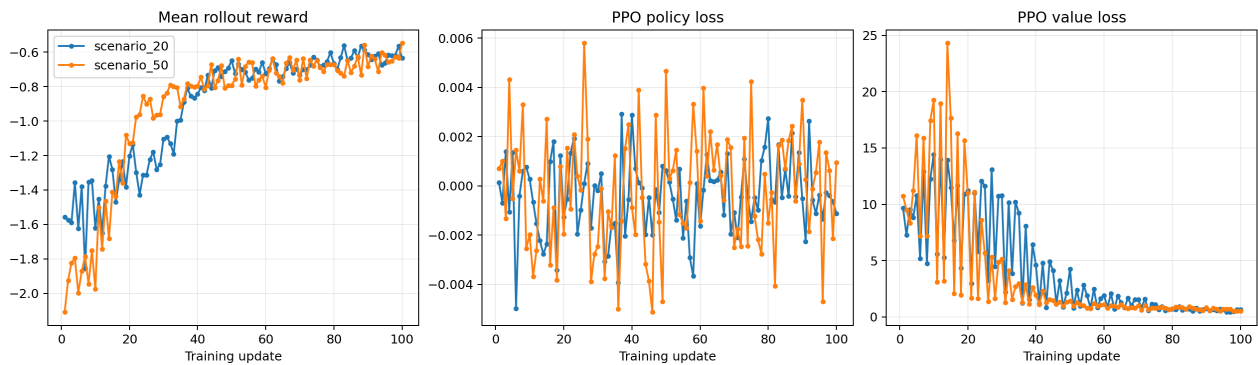
Provider occupancy heat map and mean locations — scenario 50



شکل ۳: نقشه حرارتی حضور ارائه‌دهندگان و مکان میانگین در سناریوی ۵۰/۶/۵۰

با شش ارائه‌دهنده، کنترل‌کننده محدب چند ناحیه مرکزی و شرقی پرترافیک را پوشش می‌دهد و همزمان قید متمایز بودن هدف‌ها را حفظ می‌کند.

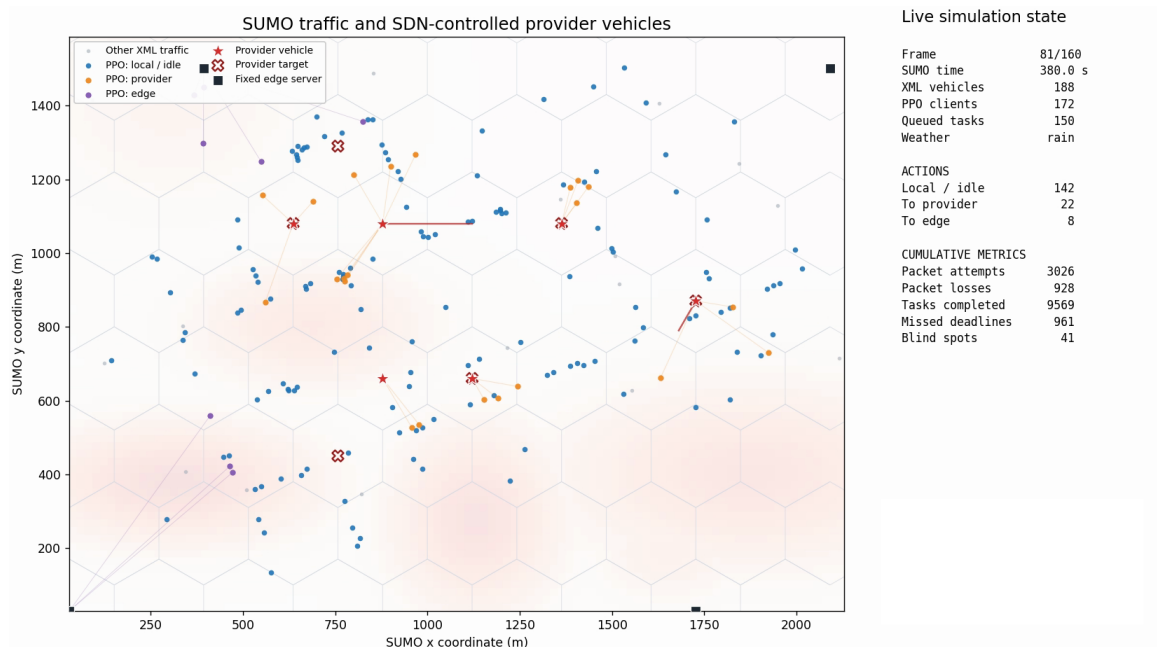
۴.۱۱ همگرایی PPO



شکل ۴: پاداش رول‌اوت، زیان سیاست و زیان ارزش در ۱۰۰ به‌روزرسانی

میانگین پاداش ده به‌روزرسانی نخست به ده به‌روزرسانی آخر در سناریوی ۲۰ خودرو از $1/526$ به $0/626$ - و در سناریوی ۵۰ خودرو از $1/898$ به $0/631$ - بهبود یافته است. زیان ارزش نیز کاهش یافته و نشان می‌دهد کریتیک مرکزی به تخمین دقیق‌تری رسیده است.

۱۲ ویدیوی گام‌به‌گام



شکل ۵: یک فریم میانی از ویدیوی ۱۷۲ مشتری و تمام ترافیک موجود در فایل

ویدیویی هم که ابتدا عمل سیاست انتخاب را مشخص می‌کند و بعد هر بار با `environment.step(actions)` پیش می‌رود، پیوست شده است.

۱۳ جمع‌بندی

مشاهده کردیم که با توجه به پیچیدگی فضای مسئله، الگوریتم یادگیری به وضوح به تخصیص‌یابی بهتر تسک‌ها و در نتیجه کاهش میس‌ها کمک کرد. همچنین با بهینه‌سازی محدب، هدف سرویس‌دهنده‌ها نیز به مراتب بهتر در فضا پخش شدند.